M.Sc. THESIS PROPOSAL

Lifecycle Security of Tool-Using AI Agents

Based on Provenance, Trust, and Authority

AUTHOR	**Parsa Oryani**
PROGRAM	**M.Sc. in Secure Computing**
INSTITUTION	**Sharif University of Technology**
EDITION	**Public English Edition · Version 1.0 · 2026**

This public edition translates and reformats the approved Persian proposal. Administrative identifiers, grades, signatures, and internal approval fields are intentionally omitted.

Proposal at a Glance

Core proposition. A tool-using agent should not be allowed to turn untrusted data into trusted instructions, temporary access into durable authority, or revoked permission into a later action. The proposed work makes those boundaries explicit and enforceable across the agent lifecycle.

Abstract

Large language model agents increasingly combine data from multiple sources, invoke tools and reusable skills, maintain long-term memory, and act through external services. These capabilities create a security problem that cannot be solved by inspecting the model's final text alone: safe behavior depends on where information came from, why it is trusted, which principal authorized an action, and whether that authority is still valid at the moment of execution.

This research proposes a model-independent lifecycle security mechanism organized around **provenance**, **trust**, and **authority**. The mechanism will mediate three connected surfaces: changes to sources, tools, and skills; admission, transformation, and retrieval of memory; and the use of information or delegated authority for externally consequential actions. It aims to prevent unjustified trust escalation, unauthorized authority creation, and reuse of revoked permissions, while producing lightweight, verifiable evidence for important decisions and actions.

A prototype will be evaluated with open-source agent frameworks under scenarios involving untrusted data, tool or skill replacement, memory poisoning, permission revocation, and out-of-authority actions. Evaluation will measure security effectiveness, normal task success, false blocking, and runtime overhead.

Research question

How can provenance, trust, and authority be represented and enforced consistently across the lifecycle of a tool-using AI agent so that untrusted inputs, mutable capabilities, and persistent memory cannot silently expand the agent's effective authority?

Keywords

Intelligent agents · agent lifecycle security · provenance · trust · authority · secure memory · runtime enforcement · tool security

Scope

- **In scope:** tool-using LLM agents, third-party skills, persistent memory, delegated permissions, runtime policy enforcement, and verifiable action evidence.
- **Out of scope:** training-time model alignment, general content moderation, and a claim of complete security outside the stated threat model.

1. Motivation and Research Gap

Why lifecycle security matters

An agent's behavior is produced by more than a model. It is shaped by system prompts, user instructions, retrieved documents, tool descriptions, skill packages, intermediate outputs, stored memories, and permissions granted by people or other agents. Each component can change over time. A system that checks only the current prompt or the final tool call therefore misses the history that explains whether an action is legitimate.

The central risk is **semantic privilege escalation**: information that began as untrusted data may later be treated as instruction; a tool description may redefine the apparent task; a memory entry may outlive the permission that created it; or a delegated capability may be reused after revocation. These failures cross component boundaries and are difficult to address with isolated guardrails.

Three security dimensions

DIMENSION	QUESTION	REQUIRED PROPERTY
Provenance	Where did this value, instruction, or capability originate, and how was it transformed?	Traceable derivation and source integrity
Trust	For which purpose, version, context, and time interval is the source or component trusted?	Explicit, scoped, non-transitive trust
Authority	Which principal may approve, delegate, narrow, revoke, or execute this action?	Least privilege and revocable delegation

Research gap

Strong recent systems address important parts of the problem: execution isolation, trusted planning, information-flow control, privilege policies, runtime rules, risk accumulation, or agent governance. The remaining gap is a single lifecycle model that preserves security meaning as artifacts move between source ingestion, tool and skill updates, memory, delegation, and external action. In particular, existing approaches do not fully explain how trust and authority should survive transformation, version change, storage, retrieval, revocation, and reuse.

Working hypothesis

If each security-relevant artifact carries a compact provenance record, an explicit trust scope, and an authority boundary - and if transitions are checked at deterministic enforcement points - then an agent can retain useful autonomy without allowing silent expansion of trust or privilege.

2. Proposed Security Model

Lifecycle surfaces

- **Source and capability lifecycle:** register, verify, version, update, replace, disable, and revoke data sources, tools, and skills.
- **Memory lifecycle:** label admission context; retain derivation through transformation and summarization; enforce retrieval and reuse constraints; invalidate dependent memories when authority or trust changes.
- **Action lifecycle:** bind each consequential action to the evidence, authority, and policy state that justified it; mediate execution at tool boundaries; record a compact decision receipt.

Core objects

OBJECT	MINIMUM SECURITY STATE
Artifact	Stable identifier, origin, version, transformation parents, integrity evidence, sensitivity, and retention state
Trust assertion	Issuer, subject, purpose, scope, justification, validity interval, and revocation state
Authority grant	Principal, delegate, permitted actions, resource and argument constraints, budget, expiry, and delegation depth
Decision receipt	Requested action, policy result, supporting artifacts and grants, enforcement point, timestamp, and outcome

Security invariants

- **No trust creation by transformation.** Summarization, retrieval, or model output cannot assign stronger trust than its justified inputs and policy allow.
- **No silent authority expansion.** Delegation and policy updates may narrow authority automatically; expansion requires an authorized approval path.
- **Revocation is transitive.** Revoking a grant or trusted component invalidates dependent capabilities, memories, and pending actions according to policy.
- **External effects require current evidence.** A sensitive action is allowed only when its provenance, trust, and authority conditions hold at execution time.
- **Important decisions are explainable.** The system records enough evidence to verify why an action was allowed, blocked, or escalated without exposing unnecessary private content.

Mechanism and enforcement

The prototype will introduce an enforcement layer outside the model. Adapters will intercept source ingestion, tool and skill registration, memory writes and reads, delegation changes, and tool calls. A policy engine will evaluate compact metadata and dependency relationships. The design will remain framework-independent at the model layer, with integration adapters for selected open-source agent runtimes.

Threat model

The adversary may control retrieved content, tool or skill metadata, selected tool outputs, or stored memory inputs; may attempt to exploit stale or over-broad permissions; and may induce the model to issue unauthorized calls. The trusted computing base includes the policy engine, integrity and identity mechanisms, the enforcement adapters, and correctly configured roots of authority. Compromise of the host operating system or cryptographic roots is outside the initial scope.

3. Methodology and Evaluation

Research method

- Formalize the lifecycle state, principals, artifacts, trust assertions, authority grants, transitions, and security invariants.
- Design enforcement semantics for updates, memory operations, delegation and revocation, and externally consequential tool calls.
- Implement a prototype with framework adapters, policy evaluation, dependency tracking, and decision receipts.
- Construct adversarial and benign scenarios on open-source agents and compare against representative baseline architectures.

Evaluation scenarios

SCENARIO	SECURITY FAILURE UNDER TEST
Untrusted content	Retrieved data is interpreted as instruction and changes a consequential action
Tool or skill replacement	A trusted name silently resolves to changed metadata, code, or capabilities
Memory poisoning	An untrusted or stale fragment is stored, transformed, and later reused as trusted context
Permission revocation	A capability, cached plan, or derived memory remains usable after its authority is withdrawn
Delegation abuse	An agent delegates beyond scope, budget, expiry, or allowed delegation depth
Out-of-authority action	The model produces a syntactically valid tool call without sufficient current authorization

Metrics

- **Security effectiveness:** prevented unsafe actions, attack success rate, and coverage of stated invariants.
- **Utility:** benign task success and completion quality under equivalent model and tool configurations.
- **Precision:** false blocking, unnecessary confirmation, and policy decision error analysis.
- **Cost:** runtime latency, metadata and storage overhead, and growth of dependency tracking.
- **Auditability:** completeness and verification time of decision receipts for selected consequential actions.

Expected contributions

- A unified vocabulary and lifecycle model connecting provenance, scoped trust, and revocable authority in tool-using agents.
- A model-independent enforcement architecture for sources, mutable capabilities, memory, delegation, and actions.
- A prototype and reproducible evaluation suite covering cross-lifecycle attacks and benign workloads.
- An empirical analysis of the security-utility-overhead trade-off and the model's limitations.

Limitations and open questions

Security will remain conditional on correct policy, trustworthy enforcement components, and accurate identification of principals and artifacts. Provenance can explain derivation without proving semantic truth; cryptographic integrity does not make a malicious source trustworthy. The work will therefore separate integrity, trust, and authority rather than collapsing them into a single score. Scalability of transitive invalidation and usability of approval prompts are explicit evaluation questions.

4. Work Plan

PHASE	ACTIVITY	DURATION
01	Review related research, systems, and evaluation methods	2 months
02	Define the precise problem, threat model, and security requirements	1 month
03	Prepare the seminar report and structured literature synthesis	1 month
04	Design the lifecycle model and proposed enforcement mechanism	2 months
05	Implement the prototype and evaluation environment	3 months
06	Evaluate, compare with baselines, and refine the design	2 months
07	Write and finalize the thesis	1 month

Total planned duration: 12 months. The work plan preserves the sequence and duration of the approved Persian proposal.

Relevant graduate coursework

- Secure Software Systems
- Formal Methods in Information Security
- Deep Learning
- Applied Cryptography

Positioning

The project sits at the intersection of systems security and agentic AI. It extends the trusted-planning, isolation, information-flow, privilege-control, and runtime-enforcement lines of work by treating provenance, trust, authority, memory, and revocation as one connected lifecycle problem.

Selected References

[1] H. Wang, C. M. Poskitt, and J. Sun, “AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents,” *48th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2026. doi: 10.1145/3744916.3764546.

[2] Y. Wu, F. Roesner, T. Kohno, N. Zhang, and U. Iqbal, “IsolateGPT: An Execution Isolation Architecture for LLM-Based Agentic Systems,” *Network and Distributed System Security Symposium (NDSS)*, 2025. doi: 10.14722/ndss.2025.241131.

[3] E. Li, T. Mallick, E. Rose, W. Robertson, A. Oprea, and C. Nita-Rotaru, “ACE: A Security Architecture for LLM-Integrated App Systems,” *Network and Distributed System Security Symposium (NDSS)*, 2026. doi: 10.14722/ndss.2026.230352.

[4] G. Syros, A. Suri, J. Ginesin, C. Nita-Rotaru, and A. Oprea, “SAGA: A Security Architecture for Governing AI Agentic Systems,” *Network and Distributed System Security Symposium (NDSS)*, 2026. doi: 10.14722/ndss.2026.230869.

[5] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, “Defeating Prompt Injections by Design,” *IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*, 2026. doi: 10.1109/SaTML68715.2026.00040.

[6] Y. Liu, Z. Chen, Y. Zhang, G. Deng, Y. Li, J. Ning, and L. Y. Zhang, “Do Not Mention This to the User: Detecting and Understanding Malicious Agent Skills in the Wild,” *35th USENIX Security Symposium*, 2026, pp. 1727-1746.

[7] Z. Rao, W. Zhu, C. A. Lu, Z. Chen, W. Niu, L. Guan, B. Li, and Z. Xiang, “FragFuse: Bypassing Access Control of Large Language Model Agents via Memory-Based Query Fragmentation and Fusion,” *35th USENIX Security Symposium*, 2026, pp. 4761-4780.

[8] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, “AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents,” *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2024.

[9] T. Shi, J. He, Z. Wang, H. Li, L. Wu, W. Guo, and D. Song, “Progent: Programmable Privilege Control for LLM Agents,” arXiv:2504.11703, 2025.

[10] X. Muruaga, “Bounded Agents: Delegation Security for Multi-Agent AI Systems,” arXiv:2608.15888, 2026.

[11] A. Catarino, C. Mamede, R. Melo, and R. M. Abreu, “TraceCaps: Inline Provenance and Risk Enforcement for Agentic Software Engineering,” *ICSE New Ideas and Emerging Results (NIER)*, 2026.

Reference policy. The public edition prioritizes peer-reviewed security and software-engineering venues. Recent preprints are included only when they directly address delegation, authority, provenance, or privilege control and are clearly labeled as arXiv work.